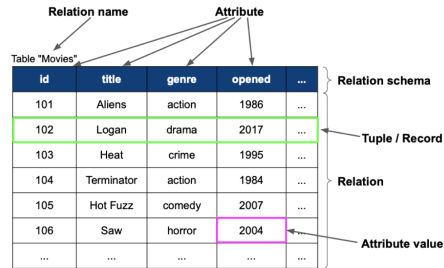


RELATIONAL DATABASE

database in which the data items are linked by internal pointers

KEYS

- **Candidate Key** = an attribute or smallest set of attributes in a table where no tuple has the same value
- **Primary Key** = a unique identifier for a table
- **Secondary Key** = a candidate key that is an alternative to the primary key
- **Foreign Key** = a set of attributes in one table that refers to the primary key in another table



Relationship

formed when one table in a database has a foreign key that refers to a primary key in another table in the database

Student ID	First Name	Second Name	Date Of Birth	Class ID
S1276	Noor	Baig	09/22/2010	7A
S1277	Ahmed	Sayed	06/11/2010	7B
S2199	Tahir	Hassan	01/30/2011	7A

↑
Class ID is the foreign key

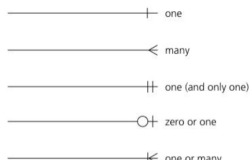
▲ Table 8.5 Part of a table for student entity

Class ID	Teacher Name	Location
7A	Mr Khan	Floor 2 Room 3
7B	Miss Malik	Floor 2 Room 4
7C	Miss Gill	Floor 2 Room 5

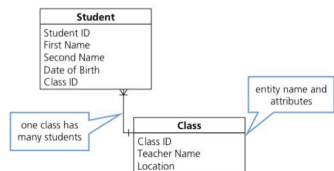
↑
Class ID is the primary key

▲ Table 8.6 Part of a table for class entity

Entity Relationship



An **E-R diagram** can be used to document the design of a database. This provides an easily understandable visual representation of how the entities in a database are related.



▲ Figure 8.3 E-R diagram for school database

NORMALISATION

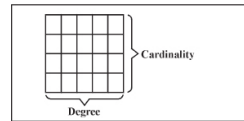
used to construct a relational database that has integrity and in which data redundancy is reduced - table that has not been normalized appear larger

Tabular Representation of Data

1. Each entry in a table occurs exactly once. (No repeating row allowed)
2. All data items in the same column are the same kind.
3. Every column has a distinct name to identify it.

There are other terms we used in the database:

- A row or a record is called a **tuple**.
- A **table** is a collection of rows.
- A **relation** is a set of tuples.
- A **degree** of a relation is the number of columns in the table.
 - Relations of degree 2 are **binary**.
 - Relations of degree 3 are **ternary**.
- The number of rows in a table is called the **cardinality** of the table.



used to construct a relational database that has integrity and in which data redundancy is reduced - table that has not been normalized appear larger

Domains

Domain is a set of all possible values for a column or a relation.

Relational Database Characteristics

- **Ordering of rows**: Row order does not matter; tables are sets of rows.
- **Ordering of columns**: Column order has no meaning from a logical standpoint.
- **Atomicity of values**: Each value must be atomic (indivisible), with no repeating values inside the same field.
- **Uniqueness of rows**: No two rows may be identical; each row must differ in at least one value.

Functional Dependency

Functional dependency is critical in formulating database design. For two columns, A and B, of a table, we can define the functional dependency between them as follows.

Column B is said to be functionally dependent on **column A** if, given A, we can precisely determine B.

First Normal Form (1NF)

entities do not contain repeated groups of attributes

Second Normal Form (2NF)

entities are in 1NF and non-key attributes depend upon the primary key

Third Normal Form (3NF)

entities are in 2NF and all non-key attributes are independent - table contains no non-key dependencies

Boyce-Codd Normal Form (BCNF)

- stronger than 3NF - has fewer redundancies
- **normal forms** in increasing order of strictness:
 - 1st NF (atomic), 2nd NF (fewer redundancies), 3NF, BCNF (always achievable), 4/5/6NF (not always achievable)
- two attributes are **functionally equivalent** if either one can determine the other

A table is in Boyce-Codd normal form (BCNF) if every column, on which some other column is fully functionally dependent, is also a candidate for the primary key of the table.

If B is functionally dependent on A, that is, A functionally determines B.

$$A \rightarrow B$$

Fourth Normal Form (4NF)

A table is in the fourth normal form (4NF) if it is in BCNF and does not have any independent multi-valued parts of the primary key.

The fourth normal form is related to the concept of a multi-valued dependency (MVD). If for a given A, there can be multiple values of B, then MVD exists.

Fifth Normal Form (5NF)

A table is in the fifth normal form (5NF), also called as project-join normal form, if it is in the fourth normal form and every join dependency in the table is implied by the candidate keys.

How to Use It

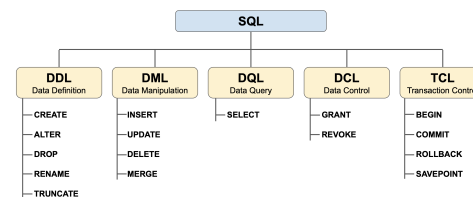
- Identify primary key.
- Check if non-key attributes depend on full key (2NF).
- Remove transitive dependencies (3NF).
- Ensure all determinants are candidate keys (BCNF).
- For multi-valued attributes → split to 4NF.

Normal Forms

- **1NF**: All values atomic.
- **2NF**: 1NF + no partial dependency.
- **3NF**: 2NF + no transitive dependency.
- **BCNF**: Every determinant is a candidate key.
- **4NF**: No independent multivalued dependency.
- **5NF**: All join dependencies implied by candidate keys.

DATABASE MANAGEMENT SYSTEMS (DBMS)

Types of Commands/Statements



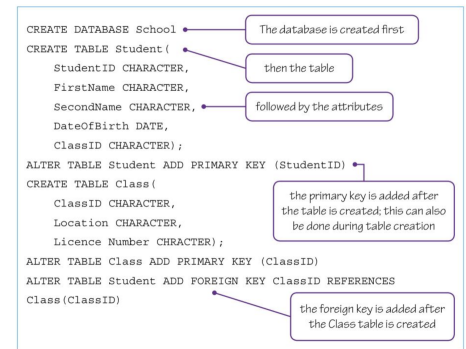
Data Types

- user-defined types
- basic data types

type	description
boolean	logical Boolean
integer	signed 4-byte integer
float8	double precision floating point number (8 bytes)
numeric(p, s)]	exact numeric of selectable precision
char(n)	fixed-length character string
varchar(n)	variable-length character string
text	variable-length character string
date	calendar date (year month day)
timestamp	date and time

- char, varchar, text: different sizes to optimise storage
 - **varchar(n)** - *n* is the maximum length
 - **char(n)** - storage size = maximum size = n (will be padded up to *n* bytes)
 - **text** - usually for very long strings

Example DDL that could have been used when the school database was create



DML commands

SQL (DML) query command	Description
SELECT FROM	Fetches data from a database. Queries always begin with SELECT.
WHERE	Includes only rows in a query that match a given condition
ORDER BY	Sorts the results from a query by a given column either alphabetically or numerically
GROUP BY	Arranges data into groups
INNER JOIN	Combines rows from different tables if the join condition is true
SUM	Returns the sum of all the values in the column
COUNT	Counts the number of rows where the column is not NULL
AVG	Returns the average value for a column with a numeric data type

SQL (DML) maintenance commands	Description
INSERT INTO	Adds new row(s) to a table
DELETE FROM	Removes row(s) from a table
UPDATE	Edits row(s) in a table

Mining Text with SQL

There are many built-in string functions that handle routine but necessary tasks, such as capitalizing letters, combining strings, and removing unwanted spaces. You may try each simple query that places a function after SELECT like

SELECT upper('hello');

Case Formatting

1. upper(string) – capitalize all alphabetical characters of a string
2. lower(string) – lowercases all alphabetical characters of a string
3. initcap(string) – capitalizes the first letter of each word. — PostgreSQL specific —

Character Information

- char_length(string) – returns the number of characters in a string, including any spaces.
- length(string) – return the number of characters in a string.
length() and char_length() may return different values when using multibyte encoding such as character sets covering Chinese, Japanese, or Korean languages.
- position(substring in string) – returns the location of the substring characters in the string.

Removing Characters

- trim(characters from string) - removes characters from the beginning and the end of a string.
- trim(string) - removes spaces at either end of the string.
- ltrim(string, characters) and rtrim(string, characters) - removes characters from the left or right ends of string

Regular Expressions – Key Patterns

Expression	Description	Expression	Description
.	A dot is a wildcard that finds any characters except a newline.	^	Match the start of a string
[FGZ]	Any character in the square brackets, for example, F, G, or z.	\$	Match at the end of a string
[a-z]	A range o characters, Here, lowercase a to z.	\?	Get the preceding match zero or one time
[^a-z]	The caret negates the match. Here, not lowercase a to z	*	Get the preceding match zero or more times.
\w	Any word character or underscore. Same as [A-Za-z0-9_]	+	Get the preceding match one or more times.
\d	Any digit.	{ m }	Get preceding match exactly m times.
\s	A space.	{ m , n }	Get the preceding match between m and n times
\t	Tab character.	a b	An alternation, find either a or b.
\n	Newline character	()	Create and report a capture group or set precedence.
\r	Carriage return character	(?:)	Negate the reporting of a capture group.

Using Regular Expression with Where

You've filtered queries using LIKE and ILIKE in WHERE clauses. With regular expression in WHERE clauses, you can perform more complex matches.

- Case-insensitive match:
WHERE column ~* 'pattern'
- Negative match:
WHERE column !~ 'pattern'
- Extract single match:
SELECT regexp_match(text, 'pattern');
- Extract multiple matches:
SELECT regexp_matches(text, 'pattern', 'g');

Regular Expression Functions to Replace or Split Text

- regexp_replace(<string>, <pattern>, <replacement_text>) lets you substitute a matched pattern with replacement text.
- regexp_split_to_table(<string>, <pattern>) splits delimited text into rows.
- regexp_split_to_array(<string>, <pattern>) splits delimited text into an array.

Example

SELECT regexp_replace('05/12/2024', '\d{4}', '2023');
SELECT regexp_split_to_table('Four,score,and,seven,years,ago', ',');
SELECT regexp_split_to_array('Phil Mike Tony Steve', '');

Views, Functions, and Triggers

Using Views to Simplify Queries

A view is essentially a stored query with a name that you can work with as if it were a table. Views are especially useful in the following scenarios:

- Avoiding duplicate effort: They let you write a complex query once and access the results when needed.
- Reducing clutter: They can trim the amount of information you need to wade through by showing only columns relevant to your needs.
- Providing security:Views can limit access to only certain columns in a table.

Create and Refreshing a Materialized View

Materialized view stored query, and the results it generates are saved in the database, which creates a new table. A good use for materialized views is to preprocess complex queries that take a while to run and make those results available for faster querying.

Create Your Own Functions and Procedures

Creating the percent_change() Function

```
percent change =  $\frac{\text{New Number} - \text{Old Number}}{\text{Old Number}}$ 

CREATE OR REPLACE FUNCTION
percent_change(new_value numeric,
               old_value numeric,
               decimal_places integer 3DEFAULT 1)
RETURNS numeric AS
'SELECT round(
((new_value - old_value) / old_value) * 100, decimal_places
);'
LANGUAGE SQL
IMMUTABLE
RETURNS NULL ON NULL INPUT;
```

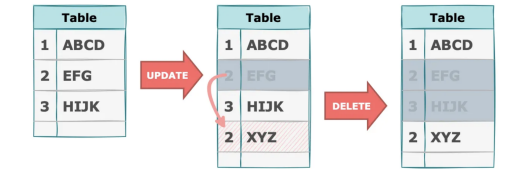
Creating the Function and Trigger

```
CREATE OR REPLACE FUNCTION record_if_grade_changed()
RETURNS trigger AS
$$
BEGIN
  IF NEW.grade <> OLD.grade THEN
    INSERT INTO grades_history (
      student_id,
      course_id,
      change_time,
      course,
      old_grade,
      new_grade)
VALUES
  (OLD.student_id,
   OLD.course_id,
   now(),
   OLD.course,
   OLD.grade,
   NEW.grade);
  END IF;
  RETURN NULL;
END;
LANGUAGE plpgsql;
```

After register the function, add the grades_update trigger to the grades table.

```
CREATE TRIGGER grades_update
AFTER UPDATE
ON grades
FOR EACH ROW
EXECUTE PROCEDURE record_if_grade_changed();
```

Maintaining Database



VACUUM Usage

- VACUUM: remove dead tuples (internal cleanup).
VACUUM table;
- VACUUM FULL: rewrite table, shrink disk file.
VACUUM FULL table;
- Autovacuum runs automatically in background.

Check Table Size

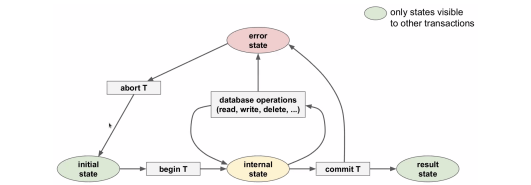
SELECT pg_size_pretty(
pg_total_relation_size('table'));

• Using pg_dump to Export a Database or Table
• Restoring a Database with pg_restore

Concurrency Control

Transactions

- transaction, T a finite sequence of database operations
 - smallest logical unit of work from an application perspective
- guarantees the ACID properties



ACID

- Atomicity → either all effects of T are reflected in the database, or none
- Consistency → the execution of T guarantees to yield a correct state of the DB
- Isolation → execution of T is isolated from the effects of concurrent transactions
- Durability → after the commit of T, its effects are permanent in case of failures

Transaction Identifier

A transaction is assigned a unique ID called the transaction identifier, or xid. PostgreSQL ensures no two active transactions share the same xid. You can view the current transaction ID with txid_current():

SELECT current_time, txid_current();

Implicit Transaction

Implicit transactions are created automatically by PostgreSQL. The system decides when the transaction begins and ends for each statement.

Explicit Transaction

Explicit transactions are controlled by the user. You start a transaction with BEGIN, and finish it with either COMMIT (save changes) or ROLLBACK (undo changes).

MVCC Multi-version concurrency control

- Writes create new tuple versions.
- Readers see snapshot → no blocking.



Isolation Levels (Usage)

- Read Uncommitted — allows a transaction to be subjected to the dirty reads problem
- Read Committed — default, avoids dirty reads.
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
- Repeatable Read — consistent snapshot.
- Serializable — safest, avoids phantoms.

Isolation Level VS Phenomena

Isolation Level	Dirty reads	Non-repeatable reads	Phantoms
Read Uncommitted	May occur	May occur	May occur
Read Committed	Don't occur	May occur	May occur
Repeatable Read	Don't occur	Don't occur	May occur
Serializable	Don't occur	Don't occur	Don't occur

Locks

- S-lock → read only.
- X-lock → read/write.

Savepoints

SAVEPOINT sp1;
ROLLBACK TO sp1;

Deadlocks

- Occur when transactions wait on each other.
- PostgreSQL auto-kills one transaction.